07 — Setup & Deployment

Prerequisites

- Python 3.11+ recommended
- Node.js 20+ / npm
- ~8 GB RAM preferred for Grounded SAM on CPU
- Disk space for model weights under `backend/storage/models/`

1. Backend (local)

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio\backend" python -m venv .venv .\.venv\Scripts\Activate.ps1 # CPU torch first, then requirements pip install torch==2.5.1 torchvision==0.20.1 --index-url https://download.pytorch.org/whl/cpu pip install -r requirements.txt # Optional: procedural textures if catalog folders are empty python scripts/generate_textures.py # One-time model download (Grounding DINO + SAM) python scripts\download_models.py copy .env.example .env # if needed uvicorn app.main:app --reload --port 8000
```

- API: http://localhost:8000
- OpenAPI: http://localhost:8000/docs

2. Frontend (local)

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio\frontend" npm install copy .env.local.example .env.local # Ensure NEXT_PUBLIC_API_BASE=http://localhost:8000 npm run dev
```

- App: http://localhost:3000

3. Docker Compose (recommended)

The app is **fully Dockerized**. Prefer this path so frontend and backend run together without separate Python/Node setup.

From project root:

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio" docker compose up --build
```

Then access the platform:

Service	URL

Platform (UI)	http://localhost:3000
Backend API docs	http://localhost:8000/docs

Compose mounts `./backend/storage` into the backend so uploads, textures, and models stay on the host.

Note: Download models once with `python scripts/download_models.py` into `backend/storage/models/` before relying on segmentation in Docker. Host machine should have about **8 GB RAM** available for the backend container.

4. Environment highlights

Backend (`.env`)

Variable	Role
`CORS_ORIGINS`	Allow frontend origin
`DEFAULT_SEGMENTATION_MODEL`	`grounded_sam`
`MODEL_CACHE_DIR`	`storage/models`
`LOG_LEVEL`	Logging verbosity

Frontend

Variable	Role
`NEXT_PUBLIC_API_BASE`	Backend origin (baked at build time for Docker)

5. Smoke test

1. Open http://localhost:3000
2. Upload a house exterior photo
3. Click Detect Elements (wait for masks)
4. Assign a paint colour → Apply finishes & render
5. Confirm before/after and cost panel
6. Download PDF

6. Folder map (runtime)

```
backend/storage/ uploads/ # ingested photos masks/ # region overlays outputs/ # rendered redesigns textures/ # cladding / tiles / patterns models/ # Grounding DINO + SAM weights brand/ # report logo assets (if present)
```

7. Cloud note (Railway)

The project includes Railway config (`backend/railway.toml`, `frontend/railway.toml`) and is **fully Dockerized**, so cloud deploy was evaluated.

Why Railway deployment was not completed

Factor	Detail
Railway free trial	Typically about **1 GB RAM** per service
This app's need	Grounding DINO + SAM on CPU need roughly **8 GB RAM** to load models and run segmentation reliably
Outcome	Services may start, but model load / Detect Elements fails or is killed under memory pressure

So we **do not proceed with Railway hosting** on the free tier. A paid plan (or any host) with ≥ **8 GB RAM** for the backend would be required before cloud deploy is practical.

What to use instead: run the Docker containers on a local machine (or any VM) that has enough RAM:

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio" docker compose up --build
```

Then open the platform at **http://localhost:3000** (API docs at http://localhost:8000/docs).

The codebase remains cloud-ready in structure (Dockerfiles + Compose + Railway toml). Scaling to Railway later only needs an instance size that can hold the ML models in memory.